UNIVERSITY OF ATHENS
DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS

Deep Learning for NLP

Student name: *Vasilakis Nikolaos*
sdi: <->

Course: *Artificial Intelligence II (M138, M226, M262, M325)*
Semester: *Spring Semester 2025*

Contents

1 Abstract **2**

2 Data processing and analysis **2**
2.1 Pre-processing . . . 2
2.2 Analysis . . . 3
2.3 Data partitioning for train, test and validation . . . 4
2.4 Vectorization . . . 5

3 Algorithms and Experiments **5**
3.1 Experiments . . . 5
3.1.1 Table of trials . . . 14
3.2 Hyper-parameter tuning . . . 14
3.3 Optimization techniques . . . 14
3.4 Evaluation . . . 15
3.4.1 ROC curve . . . 16
3.4.2 Learning Curve . . . 17
3.4.3 Confusion matrix . . . 18

4 Results and Overall Analysis **19**
4.1 Results Analysis . . . 19
4.1.1 Best trial . . . 20
4.2 Comparison with the first project . . . 21
4.3 Comparison with the second project . . . 21

5 Kaggle Notebook Links **21**

6 Bibliography **21**

1. Abstract

This is a report following the attempt to build a sentiment classifier on a given English Twitter data set. The classifier would assign tweets with a sentiment, with 1 representing a positive sentiment and 0 representing a negative sentiment. The sentiment classifier would be developed by fine-tuning two models, BERT and DistilBERT.

Moreover, each tweet in our database contained an ID, Text and a Label. The ID was used to uniquely identify the tweet, the Text was the Tweet's content, and the Label was indicating whether a tweet had either a positive or a negative sentiment.

2. Data processing and analysis

2.1. Pre-processing

Realizing that the tweets we had received, contained a lot of information, the idea of pre-processing the data with the aim of cleaning up noise was necessary. There are 6 techniques that are used to clean the data properly.

- 1. Converting all text to lowercase: This is done because the meaning of the word will be the same regardless of how many capital letters there.
- 2. Removing URLs and mentions (@s): URLs and mentions essentially give no information about the sentiment of a sentence.
- 3. Removing punctuation and non-ASCII characters: Punctuation is removed, apart from exclamation marks (!) and question marks (?) because these two can give information about the sentiment of a sentence. Fullstops, commas, etc. do not give any information about the sentiment of a sentence; therefore, they are removed. Moreover, non-ASCII characters are also removed because they do not contain relative information about a sentence and its sentiment.
- 4. Removing hash tags: Hashtags can indicate if a tweet has a positive or a negative sentiment, however, they are removed for a number of reasons. Firstly, hash tags encode words in an unstructured way (for example, they contain compound words), therefore, our model is likely to not understand the true meaning of the hash tag. Also, hash tags might cause overfitting since they the model treats hash tags as unique features and words, therefore making it much harder to generalize.
- 5. Correction of spelling mistakes: Common spelling mistakes as well as slang is corrected. For instance, the word "gud" is converted to "good", the word "wanna" is converted to "want to", etc. This is done in order to clean up some of the noise that is caused by the data.
- 6. Converting many characters into one: The final technique used, is converting more than 3 characters into a single one. For example, the word "heyyyyy" in a tweet is converted to the word "hey". This is also done in order to clean up noisy data and have a better model.

Additionally, stop word removal from common English language words such as 'am', 'or', 'and', etc. was considered, as well as words frequently found in both positive and negative tweets. While this approach was effective for models like Deep Neural Networks and Logistic Regression, BERT and DistilBERT use an attention mechanism. This mechanism allows them to understand the significance of each word and its relationship to others in the sentence. Consequently, removing any words would be unnecessary and could even detract from the contextual understanding these models gain through their attention mechanisms. Therefore, despite initial consideration, we ultimately decided against stop word removal.

2.2. Analysis

Along with the data preprocessing, an data analysis is also being done, in order to understand and vizualise the data in a better way and understand their nature.

The first data analysis that must be done, is to find out whether the data are balanced or not. A balanced data set makes the training of the model easier, because the model will not have any bias towards a particular label. Consequently, the positive and negative instances must be roughly the same.

After the analysis, it is clear that the data set is balanced with 74196 positive instances and 74192 negative instances.

Figure 1: Class distribution in the training data set

Data analysis is also used in order to find the sentence length of positive and negative sentiment tweets. This is done because it is necessary to understand how big

sentences are, and consequently adjust the 'max length' parameter when encoding the data for BERT and DistilBERT to use.

Below is the visualization of the tweet length in positive and negative instances separately.

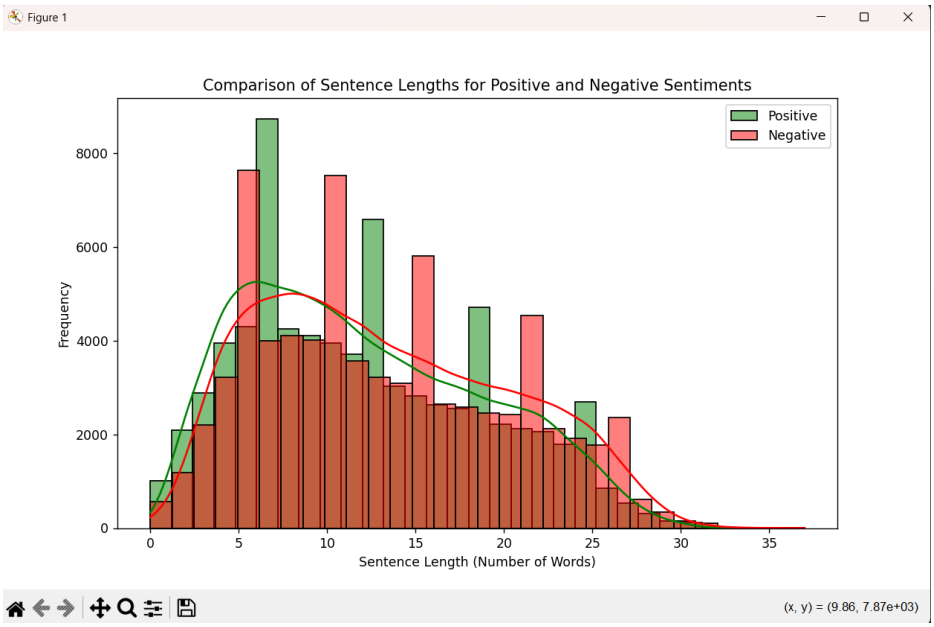


Figure 2: Sentence length for positive and negative sentiment tweets

As it is clear, most sentences are within the 5-10 word range. The longest sentences seem to be around 30 words. Taking into account how the tokenizer works (using sub-word models), it was decided that the parameter of 'max length' would be 64, as a good trade-off between computational efficiency and high accuracy of word representation.

Note: It is important to mention that the exploratory data analysis was done only on the training data. This is because we aim to avoid any potential influence from the test data, as the test set is meant for evaluating the final model's performance. Performing EDA on the test data could inadvertently lead to biases that would affect the model's generalization ability. For the same reason, no preprocessing was performed on the validation data, as it is used for hyperparameter tuning. Analyzing the validation set could have biased the selection of hyperparameters and skewed the results, leading to overfitting.

2.3. Data partitioning for train, test and validation

The data set was already partitioned when it was given to us. The size of the training and validation dataset was already well split (approximately 75% for training data set and 25% for validation data set), so there was no re-arrangement. The training data set was used to train our model and later the model was tuned based on the results given from the validation data set. No tuning or optimization was performed on the basis of the test data set. The test data set was only used for the Kaggle competition and in order to see how good is the generalization of the model.

2.4. Vectorization

To get BERT and DistilBERT ready to understand the text, we first needed to break it down into tokens. The function which helped tokenization, is 'tokenize.encode_plus' which converts words into tokens and later converts tokens into IDs that BERT can work with. For each sentence that is processed, special tokens are added as well. More specifically, a [SEP] token is added at the end of every sentence to show where it finishes, and a [CLS] token is added at the beginning of every sentence. Furthermore, all sentences are either padded or truncated to the same length (64, as explained in 2.2. Analysis), while attention masks are also added to distinguish which tokens are actual words and which are the padded ones.

3. Algorithms and Experiments

3.1. Experiments

In order to evaluate the accuracy of the classifier the training data set was used to train our model, and then the validation data set to fine tune and optimize our model. The test data set was only used to calculate the final score in the Kaggle competition and see how well the model generalized.

In order to build an efficient model, the initial approach that was used was seeing how the simplest BERT and DistilBERT models behaved and then gradually trying to increase their accuracy by fine-tuning them. The initial BERT and DistilBERT's parameters were chosen at random, and would be later fine tuned. More specifically:

- 1. AdamW was chosen as the optimizer.
- 2. Batch size was initially 16.
- 3. Learning rate was initially 2e-5.
- 4. Cross Entropy Loss was the loss function used.
- 5. 3 epochs were chosen for training

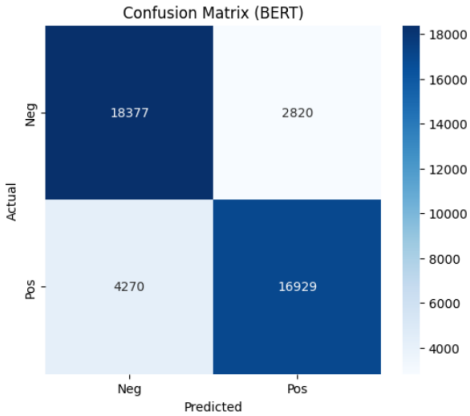
Another thing worth noting is the fact that the intial pre-processing function was the one that was used for the first two homeworks (the one which we had to develop a model based on Logistic Regression, and then the one which we had to develop a deep neural network). For this reason, in the initial pre-processing function, all the methods being said in the section 2.1 Pre-Processing are done, along with stop word removal.

Stop word removal, however, is a bad practice as we will see later, and thus results in low initial accuracy for both models, especially DistilBERT. The reason these models don't need word removal is their attention mechanism. This feature enables them to understand the importance of each word and how it connects to the rest of the sentence. Taking words out could weaken the contextual understanding that is achieved through the attention mechanism.

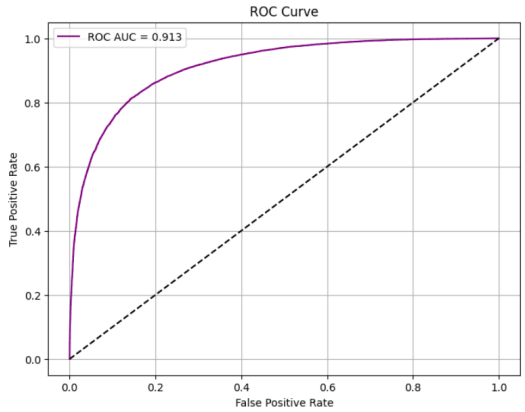
The results of the initial models vary. BERT had a high accuracy of 0.83277, while DistilBERT's accuracy was much lower, achieving only 0.78984. This is likely due to

the fact that DistilBERT has 6 layers, compared to the 12 layers that BERT has, and therefore it means that DistilBERT is not nearly as complex as the intial BERT model.

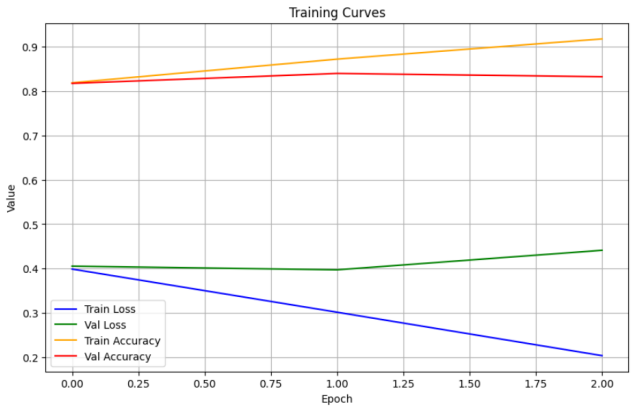
Below are the confusion matrix, the ROC curve and the learning curve for the initial BERT model:



(a) Confusion Matrix



(b) ROC Curve



(c) Learning Curve

Figure 3: Performance of the initial BERT model

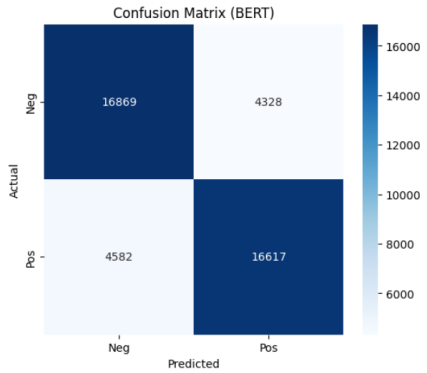
It is clear, even with just 3 epochs that the increase in the training accuracy of the model, does not match the increase in the validation accuracy of the model. Training accuracy in just 3 epochs steadily increase from approximately 0.83 to above 0.9. However, the same cannot be said for validation accuracy. Validation accuracy starts at around 0.83 and then remains mostly steady.

Consequently, it is noticeable how well the pre-trained model behaves, and even with just 3 epochs it is able to learn the training data so well, and therefore overfit the training data.

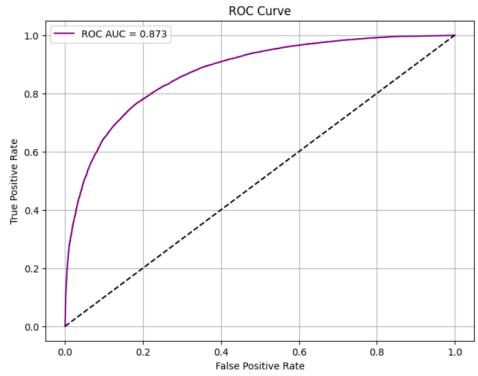
The same thing applies to DistilBERT. However, because DistilBERT has fewer layers, its initial accuracy is lower. DistilBERT starts from around 0.75 accuracy and after just 3 epochs goes above 0.80. This slower increase is likely due to the simplicity of DistilBERT compared to BERT. However, the validation accuracy remains steady in DistilBERT as well, and is barely increased after 3 epochs.

To sum up, both BERT and DistilBERT are able to overfit the training data quickly, while the validation accuracy remains mostly steady after a few epochs. So, they are able to perform relatively well on the validation data after a short amount of training and fine tuning, and further training will only increase overfitting and not have any real effect on the model's generalization.

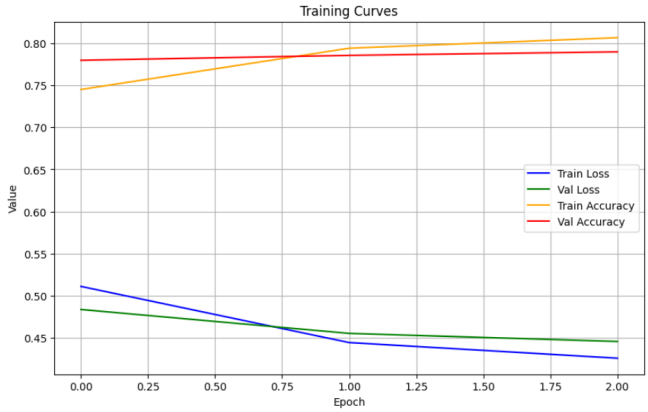
Below are the confusion matrix, the ROC curve and the learning curve for the inital DistilBERT model as well:



(a) Confusion Matrix



(b) ROC Curve

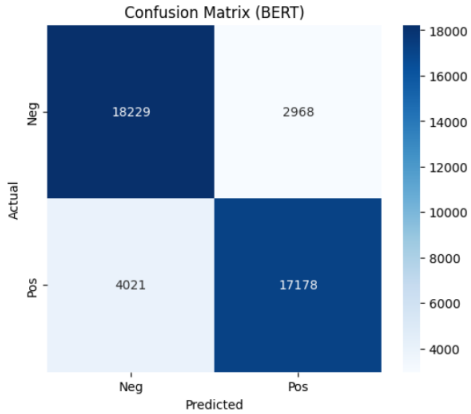


(c) Learning Curve

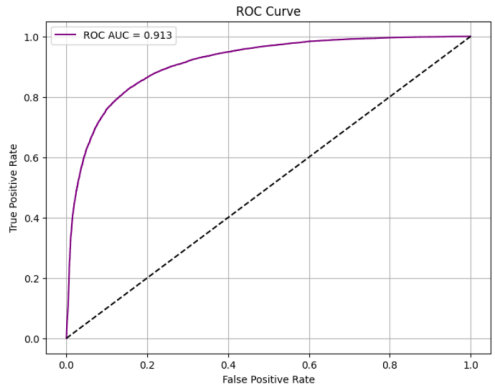
Figure 4: Performance of the initial DistilBERT model

Afterwards, the pre-processing function was improved. With the attention mechanism, the removal of stop words hurts the models instead of making them better. Because their attention mechanisms give them the ability to understand the significance of every single word and its relationships with every other word within a sentence, removing any words would be counterproductive and achieve the opposite of what the attention mechanism does. So, stop word removal would actually reduce the contextual understanding that these models have through the attention mechanism instead of helping the models achieve better accuracy. For this reason, all stop word removal was essentially not considered for the pre-processing function used.

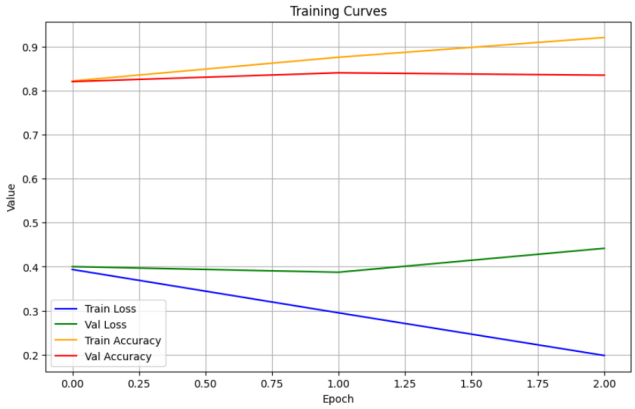
Below are the confusion matrix, the ROC curve and the learning curve for the BERT model without stopword removal:



(a) Confusion Matrix



(b) ROC Curve



(c) Learning Curve

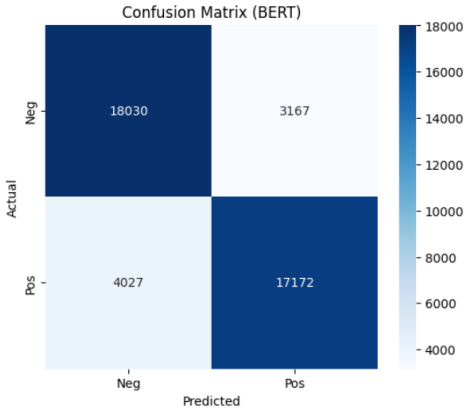
Figure 5: Performance of the BERT model without stopword removal

It is clear that without stop word removal the accuracy of the BERT model is increased from 0.83277 to 0.83515. That being said, it is clear the problem of overfitting still remains showing that the model is able to learn training data very quickly.

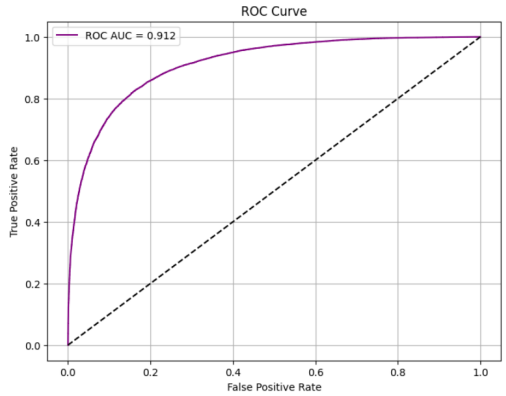
However, not having stop word removal still remains useful, as it increases accuracy. The effect on DistilBERT is much more impressive. Since DistilBERT has only 6 layers instead of 12, the adjustment of the pre-processing function has a greater effect. In fact, without stop word removal, DistilBERT's accuracy goes from 0.78984 to 0.83031, showing above a 4% increase in accuracy. This indicates the greatness of the attention mechanism. A model with only 6 layers is able to learn language patterns and behaviors quickly. Even removing a few words like 'the', 'and', 'or, and 'of', contradicts the model's learned capacity because the removal of stop words is contradictory to the attention mechanism. In fact, with the removal of stop words there is above a 4% accuracy drop, showing how powerful this mechanism actually is.

The problem of overfitting still remains though and it appears that both models quickly learn the training data.

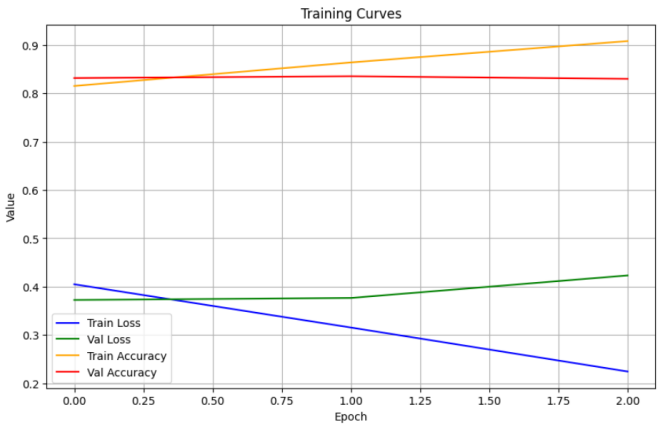
Below are the confusion matrix, the ROC curve and the learning curve for the DistilBERT model without stopword removal:



(a) Confusion Matrix



(b) ROC Curve



(c) Learning Curve

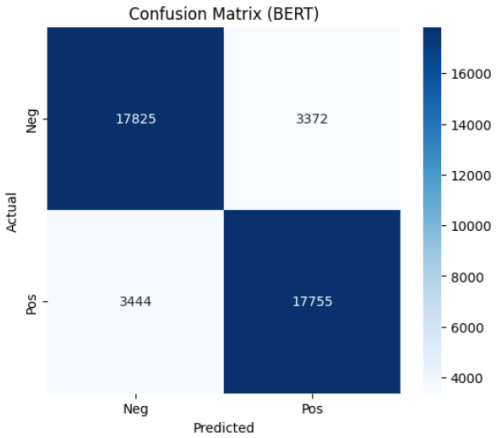
Figure 6: Performance of the DistilBERT model without stopword removal

Continuing with the construction of a more complex BERT and DistilBERT model, early stopping and the addition of a scheduler were later used. This was done to combat the severe overfitting which occurred after a few epochs. In fact, if accuracy did not improve after 2 epochs, early stopping was triggered, meaning that the training would be cut-off. The reason only 2 epochs were chosen was because severe overfitting was noticed after only a few epochs so it needed to be cut off fast.

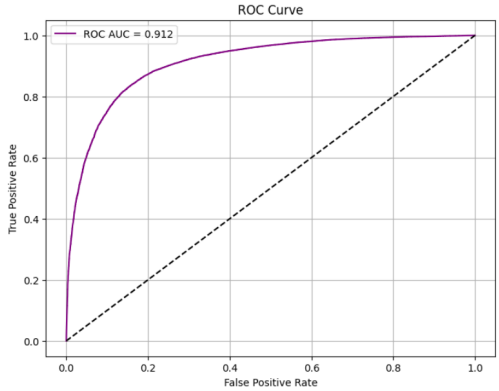
Additionally, the scheduler that was used was the one called 'get linear schedule with warmup'. What the scheduler essentially did was to start with the regular learning rate that was set and slowly decrease it all the way down to 0 over each training step in order to make training even more stable.

Furthermore, since early stopping was added, the number of epochs was increased from 3 to 4 in order to see if early stopping had any beneficial results and what the effects of the epoch increase and the addition of the scheduler would be

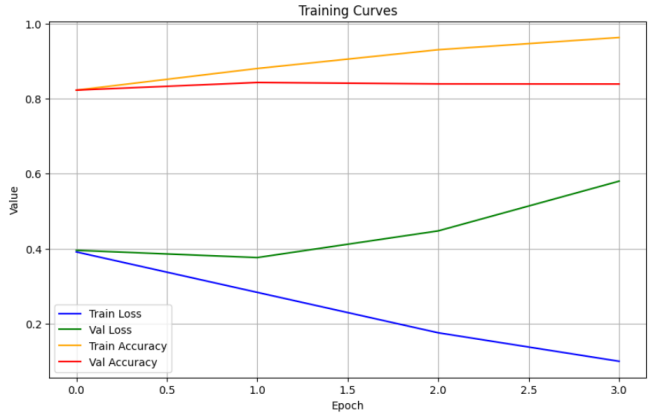
Below are the confusion matrix, the ROC curve and the learning curve for the BERT model with the addition of a scheduler and early stopping:



(a) Confusion Matrix



(b) ROC Curve



(c) Learning Curve

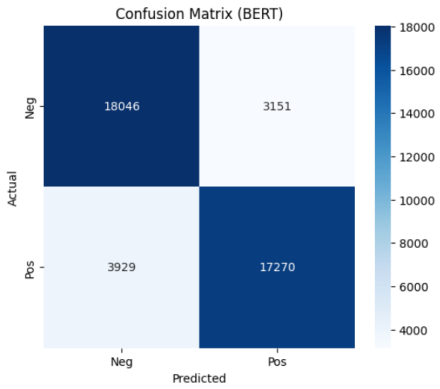
Figure 7: Performance of the BERT model with the addition of a scheduler and early stopping

As it is clear, the addition of 4 epochs does extreme harm to overfitting and the generalization of the model. BERT learns the data so quickly that even early stopping is not able to stop it. However, accuracy is still increased from 0.83515 to 0.83923, possibly due to the addition of the scheduler, as training becomes smoother and more stable.

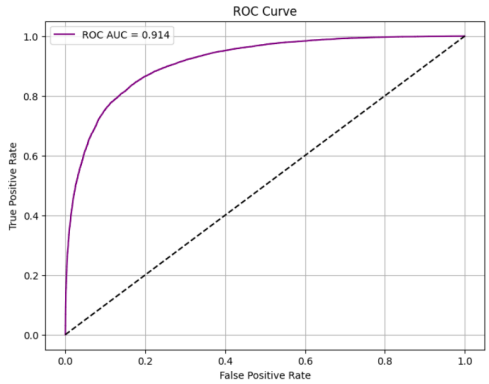
On the other hand, on DistilBERT, early stopping is triggered after the third epoch, indicating that the model does not improve after the first or second epoch, meaning that it learns training data even faster than BERT. One possible explanation is that because of the 6 layers of DistilBERT, it is not able to generalize as well as BERT. It is noticeable that accuracy is also increased on DistilBERT from 0.83031 to 0.83300, again probably due to the addition of the scheduler (as BERT).

Consequently, it is clear that regardless of early stopping, adding a scheduler helps the model to improve its accuracy. Moreover, having early stopping helps the model to combat overfitting - like in the case of DistilBERT which is triggered after only 3 epochs.

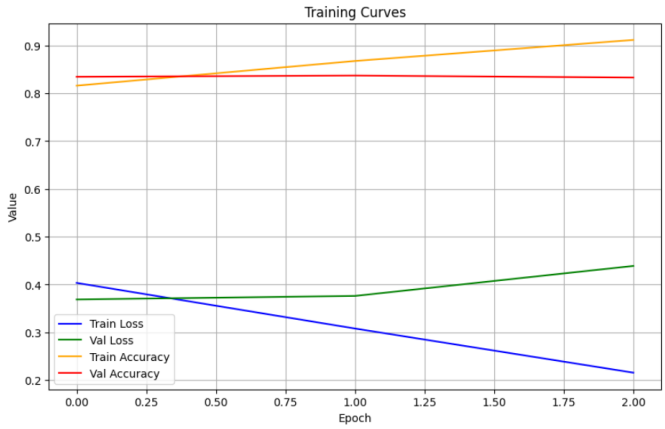
Below are the confusion matrix, the ROC curve and the learning curve for the DistilBERT model with the addition of a scheduler and early stopping:



(a) Confusion Matrix



(b) ROC Curve



(c) Learning Curve

Figure 8: Performance of the DistilBERT model with the addition of a scheduler and early stopping

Later on, hyper-parameter tuning was necessary in order to fine tune the hyper-parameters suggested in the tutorial. These parameters included the batch size, the learning rate and the epoch count. The optimizer (Adam or AdamW) was also tested.

After the previous results, it was clear that the epoch count needed to be reduced to a low number (1 or 2), however, more numbers were tested for clarity and robustness.

For the purpose of hyper-parameter tuning, the tool 'Optuna' was used in order to find the best possible accuracy and the lowest validation loss. The values tested on 'Optuna' were:

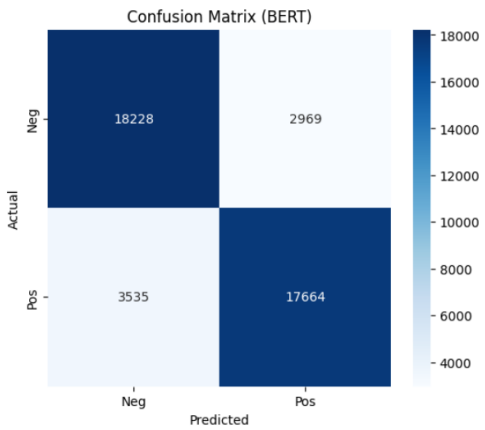
- 1. Batch Size (16 or 32) - as suggested in the tutorial
- 2. Learning Rate (5e-5, 3e-5, 2e-5) - as suggested in the tutorial
- 3. Epoch Count (2, 3 or 4) - as suggested in the tutorial

- 4. Optimizer (Adam or AdamW)

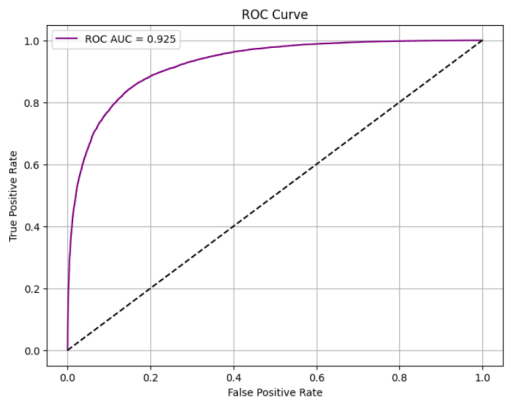
After running 'Optuna', a batch size of 32, an epoch count of 2, the Adam optimizer and a learning rate of 2e-5 were chosen for the best possible accuracy.

After the hyper-parameter tuning, accuracy saw an approximate increase of more than 0.5% from 0.83923 to 0.84659.

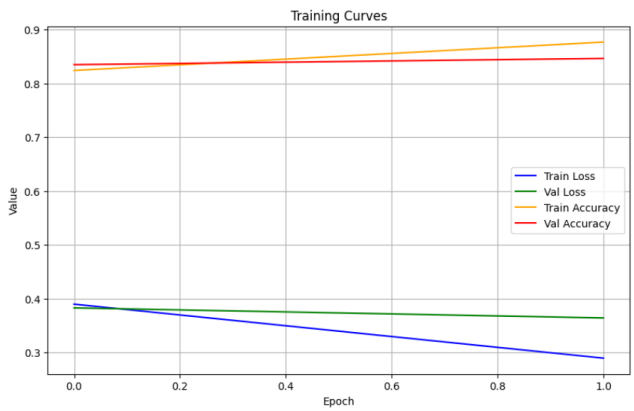
Below are the confusion matrix, the ROC curve and the learning curve for the BERT model after the hyper-parameter tuning:



(a) Confusion Matrix



(b) ROC Curve



(c) Learning Curve

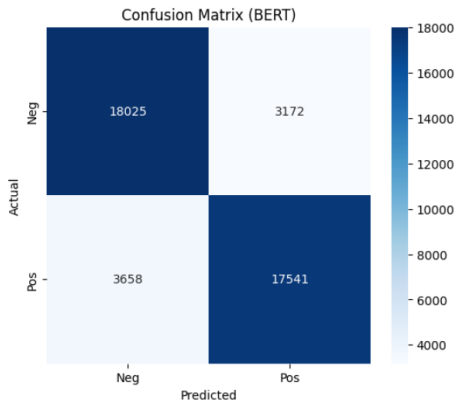
Figure 9: Performance of the BERT model after the hyper-parameter tuning

A different 'Optuna' trial was run on DistilBERT as it is a different model with fewer layers and fewer parameters. The values tested on the 'Optuna' trial were the same as the ones for the BERT model.

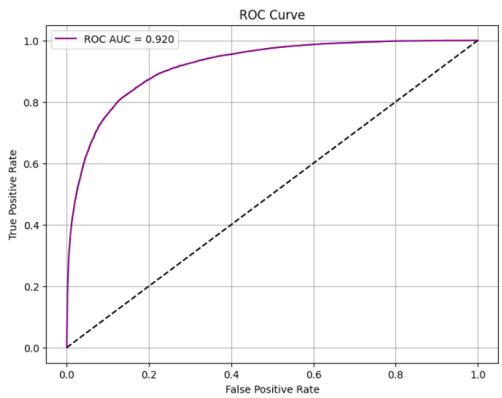
After running 'Optuna', a batch size of 32, an epoch count of 2, the Adam optimizer and a learning rate of 3e-5 were chosen for the best possible accuracy. So, mostly the same parameters as BERT were chosen with the exception of the learning rate (3e-5 instead of 2e-5).

After the hyper-parameter tuning, accuracy saw an increase of approximately 0.6% from 0.83300 to 0.83890.

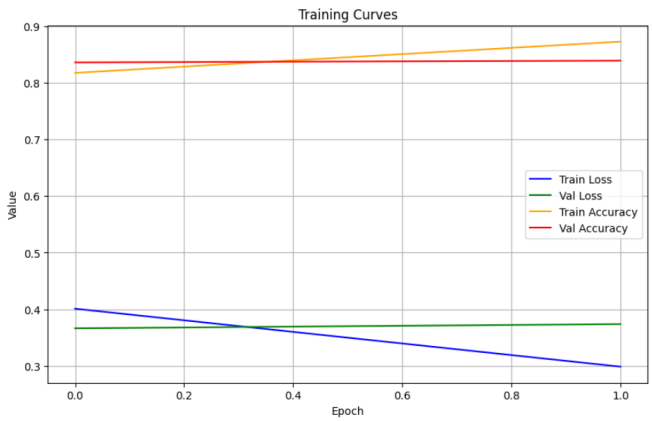
Below are the confusion matrix, the ROC curve and the learning curve for the DistilBERT model after the hyper-parameter tuning:



(a) Confusion Matrix



(b) ROC Curve



(c) Learning Curve

Figure 10: Performance of the DistilBERT model after the hyper-parameter tuning

A few unsuccessful experiments were also tried, which ultimately failed for different reasons each:

1. Another experiment was switching from an "uncased" to a "cased" model. Using the cased model actually made the accuracy worse, possibly because of the noise of the tweets. The meaning of a tweet is the same regardless of lowercase or capital letters, so with the cased model, noise was added making the model to react less well to the tweets. In fact, the the accuracy was dropped from 0.84659 to 0.84253.

This change had similar results in DistilBERT with the accuracy dropping by nearly 0.5%, from 0.83890 to 0.83531.

2. Another change was to try and increase dropout rates for both BERT and DistilBERT. Manual tuning was chosen instead of doing a full 'Optuna' trial, because only 2 parameters needed to be tuned and because of the large computational cost that an 'Optuna' trial would have. The results were surprising for both BERT and DistilBERT.

Initially, both hidden dropout and attention dropout values were increased. However, it was found that the more they were increased them from the default setting of 0.1, the worse the accuracy got (even almost 1% worse when dropout values were around 0.3). On the contrary, the generalization gap was smaller for higher dropout values. Thus, the default value of 0.1 was kept, as we wanted a higher accuracy and also because it was considered a good balance and a good trade-off between performance and the ability of both models to generalize well.

3. In contrast, lowering the dropout and attention dropout values had very different results. While accuracy slightly increased for DistilBERT (from 0.83890 to 0.83954), it was decreased for BERT (from 0.84659 to 0.84624). This also came at the cost of a significantly larger generalization gap. Given these minimal changes in accuracy versus the worsened generalization, the default dropout values of 0.1 were ultimately selected as they represented a good balance and a good trade-off between performance and the ability to generalize.

3.1.1. Table of trials. Below we show the table of trials that were done in order to improve our model. A synopsis of the trial is presented alongside the accuracy score for each model.

Note: This is table is only a synopsis of the trials, as full analysis was given above as well as in sections 3.2 and 3.3 later on.

Trial	BERT Score	DistilBERT Score
Initial Model	0.83277	0.78984
Model without stopword removal	0.83515	0.83031
Adding early stopping and a scheduler	0.83923	0.83300
Hyperparameter tuning	0.84659	0.83890
Changing to a cased model	0.84253	0.83531
Increasing dropout rates	0.84480	0.83753
Lowering dropout rates	0.84624	0.83954
Final model	0.84659	0.83890

Table 1: Summary of trials comparing BERT and DistilBERT scores

3.2. Hyper-parameter tuning

To better tune the model, the hyper-parameters needed to be tuned. The tool 'Optuna' was used in order to tune the hyper-parameters suggested by the tutorial. All the internal layers of BERT were kept frozen and only the learning rate, epoch count, optimizer and batch size were tuned. After running 'Optuna' for all possible combinations, the best parameters were found for both models, as said above (in 3.1. Experiments). The best parameters were the same for BERT and DistilBERT (Adam optimizer, batch size of 32 and epoch count equal to 2), apart from the learning rate which was 2e-5 for BERT and 3e-5 for DistilBERT.

3.3. Optimization techniques

To optimize our model apart from hyper-parameter tuning the following were used:

1. Early Stopping: Early stopping was used in order to stop training the model if it did not improve its results over a few epochs. This helped avoid overfitting and also gave the model the ability to reach its best performance.

2. Learning Rate Scheduler: Finally, adding a learning rate scheduler helped to change the learning rate . With a scheduler, the regular learning rate that is initially set, slowly decreases all the way down to 0 over each training step in order to make training even more stable.Also, weight updates are done smoother and convergence is better.

3. 'Optuna' Hyper-parameter tuning: Finally, the tool 'Optuna' was used to fine-tune the hyper-parameters and give them the best possible values. Learning rate, optimizer, batch size and epoch count were tuned.

All these techniques, lead to an optimizer model with a good ability to discriminate between positive and negative sentiments, and with an accuracy of about 0.84659 for BERT and 0.83890 for DistilBERT.

3.4. Evaluation

To evaluate the performance of the model, the evaluation metric that is used is accuracy. Since our dataset is balanced, and there is no priority of false negatives over false positives (or vice versa) as both types of misclassification have similar consequences, accuracy is the preferred metric.

The scores of recall, precision and F1-score are given below:

```
BERT Evaluation:
Accuracy: 0.84659
Precision: 0.85610
Recall: 0.83325
F1 Score: 0.84452
              precision    recall  f1-score   support

           0       0.84      0.86      0.85     21197
           1       0.86      0.83      0.84     21199

    accuracy                           0.85     42396
   macro avg       0.85      0.85      0.85     42396
weighted avg       0.85      0.85      0.85     42396
```

Figure 11: Different evaluation metrics for the final BERT model

The above results show the precision-recall trade-off. The F1-score is a balance between these two metrics. Since the dataset is balanced, accuracy is also close to the F1-score, reflecting the overall performance without being skewed by class imbalance.

```
BERT Evaluation:
Accuracy: 0.83890
Precision: 0.84686
Recall: 0.82744
F1 Score: 0.83704
              precision    recall  f1-score   support

           0       0.83      0.85      0.84     21197
           1       0.85      0.83      0.84     21199

    accuracy                           0.84     42396
   macro avg       0.84      0.84      0.84     42396
weighted avg       0.84      0.84      0.84     42396
```

Figure 12: Different evaluation metrics for the final DistilBERT model

DistilBERT has a lower accuracy of approximately 1% likely due to the fact that it is a model with fewer layers and fewer parameters.

3.4.1. ROC curve. Another useful way to evalute our model is through the ROC curve. The receiver operating characteristic (ROC) curve is a tool used for evaluating the performance of a binary classifier. The ROC curve plots the true positive rate against the false positive rate at different thresholds

The ROC curves of the final models are given below:

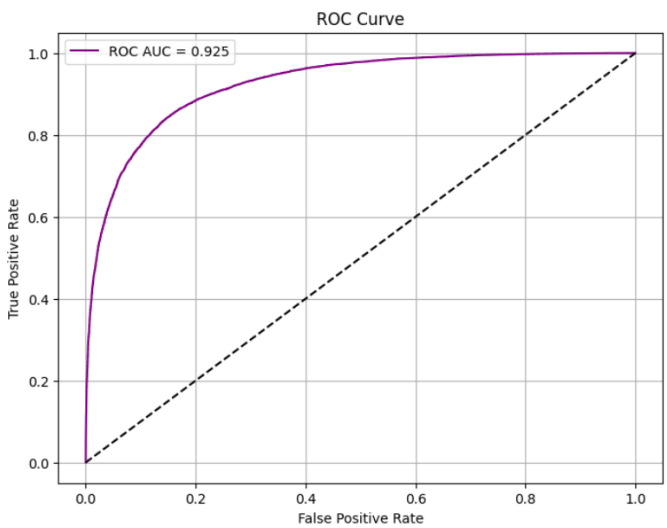


Figure 13: ROC curve for the final BERT model

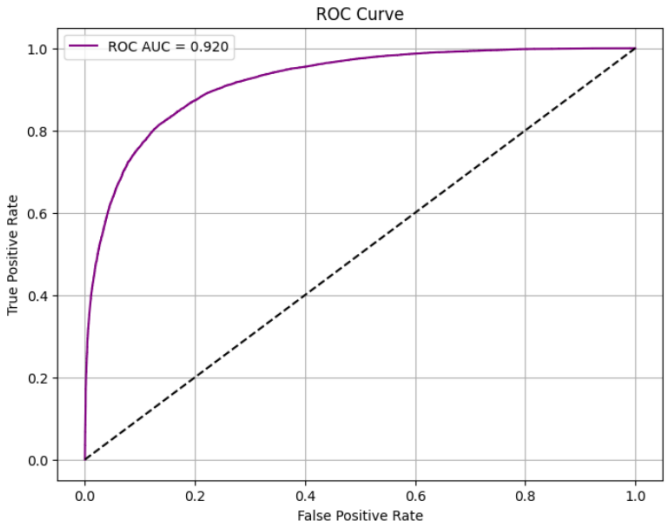


Figure 14: ROC curve for the final DistilBERT model

The ROC curve has an Area Under Curve (AUC) of 0.925 for the BERT model, and an Area Under Curve (AUC) of 0.920 which shows a good ability to discriminate between positive and negative sentiments, compared to a random classifier for both of these models.

3.4.2. Learning Curve. The learning curves are another good way to evaluate the final model. A learning curve shows the performance of the model against training size. Four curves are shown simultaneously. Two curves that concern the training performance(training accuracy and training loss) and two curves that concern the validation performance(validation accuracy and validation loss).

Below are the learning curves for the final models.

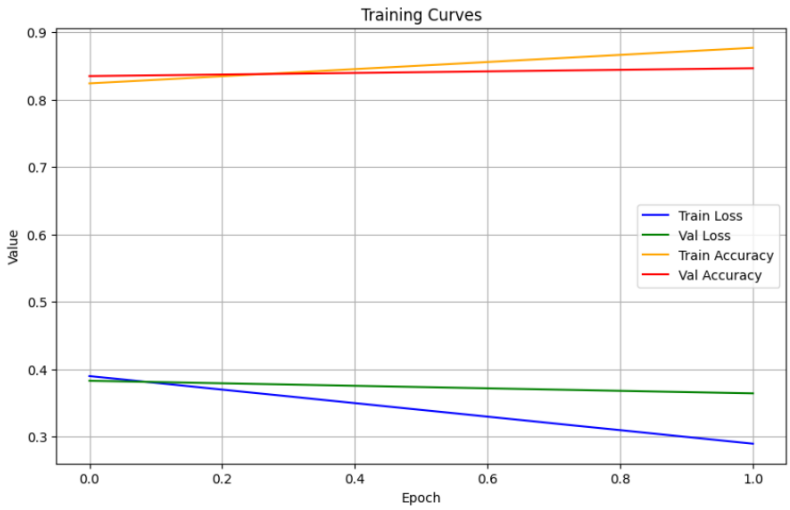


Figure 15: Learning curves for the final BERT model

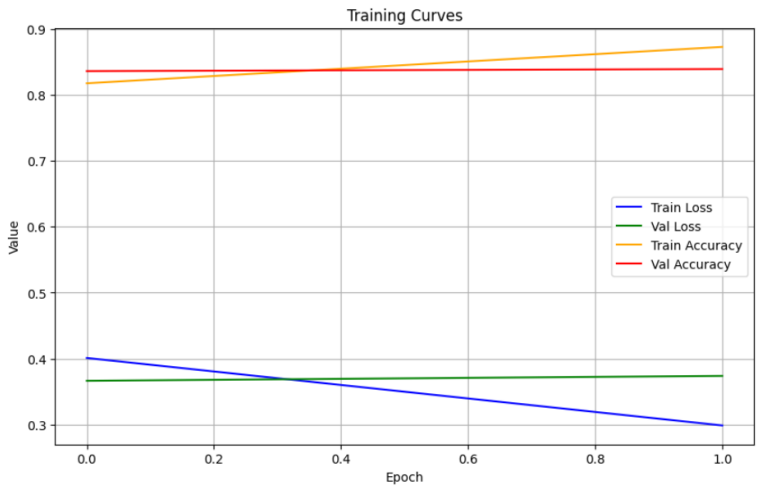


Figure 16: Learning curves for the final DistilBERT model

From the learning curves, it is clear that there is overfitting, even with just two epochs as the training and validation accuracy curves are becoming more distant from each other. The training accuracy curve shows an upward trend, while the validation accuracy curve is more stable. With more epochs, there are large signs of overfitting, and this is the reason that only two epochs are used in training. Similarly, the training and validation loss curves are becoming more distant from each other. The training loss curve shows a downward trend, while the validation loss curve is stable.

3.4.3. Confusion matrix. Finally, a better way to evaluate the performance of our model is the confusion matrix. The confusion matrix works like this: Each row of a confusion matrix represents an actual class, while each column represents a predicted class. Therefore, true positives, true negatives, false positives and false negatives all have a numerical value in the confusion matrix. Therefore, a confusion matrix gives a vizual representation of metrics beyond accuracy such as recall and precision and is useful in identifying possible problems with our model.

The confusion matrix for the BERT model shows that the number of False Positives is lower than the number of False Negatives. As a result, the recall score is slightly lower than the precision score (approximately around 2%). However, since the True Positives and True Negatives are significantly higher overall, the model demonstrates high accuracy.

The confusion matrix shows for the DistilBERT model shows similar behaviour. The recall score is slightly lower than the precision score (again approximately around 2%). However, since the True Positives and True Negatives are significantly higher overall, the model demonstrates high accuracy.

Below are the confusion matrices of the final models.

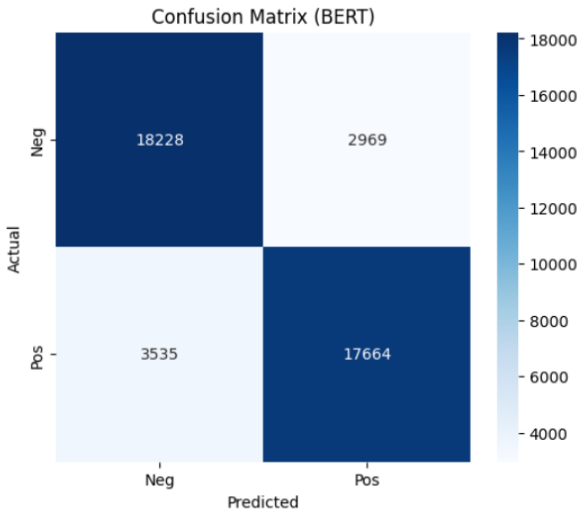


Figure 17: Confusion matrix for the BERT final model

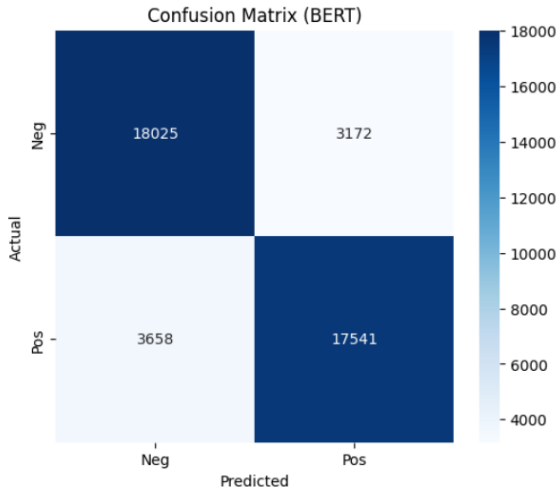


Figure 18: Confusion matrix for the DistilBERT final model

4. Results and Overall Analysis

4.1. Results Analysis

The final BERT model scores an accuracy of 0.84659, while the final DistilBERT model scores an accuracy of 0.83890. This shows that both models have a good ability to generalize well and can correctly classify most instances. The 1% difference between BERT and DistilBERT is likely due to DistilBERT's simplicity and the fact that it has much fewer parameters and fewer layers also. Nonetheless, it is impressive that a model like DistilBERT almost reaches BERT's accuracy while being much smaller.

One of the most impactful changes to our models was the removal of stop words. The removal of stop words was damaging the attention mechanism of both BERT and DistilBERT, and therefore, did not allow for every word to understand its contextual

meaning when compared with other words in the same sentence. The removal of stop words appeared to confuse the models more, instead of helping them, especially DistilBERT. When stop words were not removed DistilBERT saw a huge increase in accuracy of around 4%.

The learning curve and ROC curve show that the classifier is much better at separating positive from negative sentiments on tweets compared to a random one, achieving an accuracy of 0.84659 for BERT and 0.83890 for DistilBERT. Moreover, both models have signs of overfitting, even with only 2 epochs used for the training process. Hence, it is clear that they have a good ability to generalize well without extra training, and training them only for 1-2 epochs is enough to get good results. This is an indicator of how powerful both of the pre-trained models are, since they can get good results with minimal fine tuning.

One more thing to add is that fact that it would be ideal for all wrong spellings and slang to be replaced with the correct words, in order for less noise to occur in the training data. However, it is essentially impossible to include in the pre-processing function every possible wrong spelling or every possible slang that would be used in a tweet. Therefore, only common misspellings and common slang were replaced.

Other changes such as switching to a cased model and adjusting the attention dropout and hidden dropout rates did not improve the model.

In conclusion, the gradual change of the model, from the intial model and then adjusting the pre-processing function, adding early stopping, adding a scheduler and having hyperparameter tuning led to much better results and we ended up with robust final models.

All in all, it is clear that mostly the same parameters and experiments apply to both models. It is impressive that even though DistilBERT has fewer parameters and hidden layers, it has an accuracy of around 1% lower than BERT. Both models behave mostly in the same way and are much stronger than a simple linear classifier or a simple neural network.

4.1.1. Best trial. BERT's best trial achieved an accuracy of 0.84659 while DistilBERT's best trial achieved an accuracy of 0.83890. While the initial models were effective without fine-tuning them, a few changes were made to make them more effective.

The best model consists of techniques such as early stopping, and the addition of a scheduler to dynamically adjust the learning rate. Moreover, the tool 'Optuna' was used to fine-tune batch size, learning rate, epoch count and the optimizer.

BERT has Adam as its optimizer, a batch size of 32, a learning rate of 2e-5, and an epoch count equal to 2.

DistilBERT has mostly the same hyper-parameters as BERT. Adam is its optimizer, batch size is 32 and epoch count is 2. The only difference is its learning rate which is 3e-5 instead of 2e-5.

Moreover, a big part in making the model effective was the pre-processing function which included converting every letter to lowercase, correcting common spelling mistakes and converting 'slang' terms to their actual meaning. Additionally, removing URLs, mentions, hashtags, non-ASCII characters and punctuation (except for exclamation marks and question marks) increased accuracy. Converting more than 3 characters into one (e.g. converting 'heyyyy' into 'hey') was beneficial as it reduced noise and cleaned the data.

4.2. Comparison with the first project

The first and third project had different results with more than 4% percent difference for the BERT model, while with more than 3% percent difference for the DistilBERT model. The accuracy in the first project with the linear classifier was around 0.80373, while the accuracy in the third project with BERT was around 0.84659. DistilBERT's accuracy was also an improvement - around 0.83890. This is likely due to the fact that the attention mechanism that both BERT and DistilBERT models have can understand language patterns and context much better than a simple linear classifier. Furthermore, both BERT and DistilBERT are pre-trained with much more parameters than the simple linear classifier, thus being able to have a greater accuracy with only a few training epochs.

4.3. Comparison with the second project

The third project achieved a much better performance than the second one. The deep neural network achieved an accuracy of 0.78394 while BERT's accuracy was improved by over 6% to 0.84659. Even DistilBERT's accuracy was increased by almost 5.5% which is a much smaller model than BERT. This shows the power of the attention mecahnism and how well these pre-trained models can perform on a classification task.

5. Kaggle Notebook Links

Below are the links for the Kaggle notebooks:
For BERT: https://www.kaggle.com/code/sdi2200016/sdi2200016-bert
For DistilBERT: https://www.kaggle.com/code/sdi2200016/sdi2200016-distilbert

6. Bibliography

References

[1] Classification: Roc and auc, 2024. https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc.

[2] Vyacheslav Efimov. Large language models: Distilbert, 2023. https://towardsdatascience.com/distilbert-11c8810d29fc/.

[3] Tatsunori Hashimoto. Lstm rnns and neural machine translation, 2025. https://web.stanford.edu/class/cs224n/slides_w25/cs224n-2025-lecture06-fancy-rnn.pdf.

[4] Manolis Koubarakis. Artificial intelligence ii course homepage for the department of informatics and telecommunications, 2025. https://cgi.di.uoa.gr/~ys19/.

[5] Manolis Koubarakis. Bert, 2025. https://cgi.di.uoa.gr/~ys19/lectures25/BERT.pdf.

[6] Manolis Koubarakis. Transformers, 2025. https://cgi.di.uoa.gr/~ys19/lectures25/transformers.pdf.

[7] Despina-Athanasia Pantazi. Tutorial for bert, 2025. https://cgi.di.uoa.gr/~ys19/tutorials25/AI_II_Tutorial_BERT_Word_Embeddings.ipynb.

[8] Despina-Athanasia Pantazi. Tutorial for bert word embeddings, 2025. https://cgi.di.uoa.gr/~ys19/tutorials25/AI_II_tutorial_BERT_Fine_Tuning.ipynb.

You can refer to the material from [1], [2], [3], [4], [5], [6], [7], and [8].